

Segmentation de tissus cérébraux sur IRM

Dataset iSeg-2017

Hackathon SCIA 2026

4 septembre 2026

1 Introduction

Ce projet vise à segmenter des IRM cérébrales de nourrissons en trois tissus d'intérêt (liquide céphalo-rachidien, matière grise, matière blanche) à partir du dataset public *iSeg-2017*. Ce rapport présente le dataset, les choix de conception retenus pour préparer les données, et un problème identifié en cours d'exploration : la présence massive de tranches totalement vides dans le volume, ainsi que la solution mise en place pour pouvoir les exclure et en mesurer l'impact.

2 Dataset

Le dataset iSeg-2017 fournit, pour chaque sujet, deux volumes IRM 3D (modalités T1 et T2) ainsi qu'un volume de vérité terrain associant à chaque voxel l'une de quatre classes : fond, liquide céphalo-rachidien (LCR), matière grise et matière blanche. Les fichiers sont au format Analyze 7.5 (paires `.hdr/.img`).

Le dataset est scindé en deux ensembles :

- **Entraînement** : 10 sujets, avec label de vérité terrain ;
- **Test** : 13 sujets, sans label (réservés à l'évaluation externe du challenge iSeg-2017).

Chaque volume 3D peut être découpé en tranches 2D selon l'un de ses trois axes ; les tranches obtenues selon des axes différents n'ont pas nécessairement la même taille, ce qui impose une mise à l'échelle commune avant de pouvoir les empiler dans un même tenseur.

3 Choix de conception

Le projet a été structuré autour d'un pipeline en trois étapes indépendantes du choix de segmentation proprement dit :

- **Préparation** : les archives brutes du challenge sont extraites et réorganisées en un dossier par sujet (`dataset/train/subject-n` et `dataset/test/subject-n`), séparant clairement les sujets avec et sans label.
- **Chargement** : les volumes sont découpés en tranches 2D selon l'axe choisi, puis mis à l'échelle vers une taille carrée commune, soit par remplissage (padding), soit par interpolation. Le label étant catégoriel, son interpolation utilise systématiquement le mode « plus proche voisin » pour ne jamais créer de classe inexistante aux frontières entre tissus. Les valeurs brutes du label sont ensuite remappées vers des indices de classe contigus, directement exploitables par une fonction de coût standard de classification.
- **Chargement PyTorch** : les tranches sont exposées via un `Dataset/DataLoader` PyTorch classique, paramétré par l'axe de coupe et la modalité (T1 ou T2) utilisée en entrée.

Un module de visualisation permet d'afficher une tranche et son label légendé par classe, ainsi que de reconstruire et afficher en 3D (via marching cubes) les surfaces de segmentation d'un volume complet, que ce soit à partir de la vérité terrain ou d'une prédiction de modèle.

Ces choix ne présument pas encore de l’architecture de segmentation elle-même, qui reste à définir ; ils fixent seulement la manière dont les données sont rendues exploitables en amont.

4 Tranches vides : constat et solution

En explorant le dataset, nous avons constaté qu’une grande partie des tranches extraites sont totalement noires : ce sont les coupes prises aux extrémités du volume 3D, en dehors de la zone occupée par le cerveau, où l’image comme le label ne contiennent que du fond. Ces tranches n’apportent aucune information utile à l’entraînement et risquent au contraire de le biaiser, le modèle pouvant apprendre à prédire le fond partout pour minimiser trivialement l’erreur.

Une mesure sur l’ensemble d’entraînement (10 sujets, découpage selon un seul axe, soit 2560 tranches au total) confirme l’ampleur du problème : **1550 tranches sur 2560 (environ 60 %) sont entièrement dépourvues de tissu segmenté**, et ce de façon homogène d’un sujet à l’autre (entre 58 % et 64 % de tranches vides selon les sujets).

Pour pouvoir retirer ces tranches au besoin, et plus généralement doser la part de tranches peu remplies conservées, nous avons ajouté au chargement des données un seuil configurable : une tranche n’est gardée que si la fraction de ses pixels appartenant à un tissu (donc hors fond) atteint ce seuil. Fixé à 0, ce seuil ne filtre rien ; augmenté, il élimine d’abord les tranches vides, puis progressivement les tranches les moins informatives.

Cette mesure suit une distribution continue plutôt qu’un simple partage vide/non-vide : le nombre de tranches conservées chute très vite dès les premiers seuils, avant de se stabiliser. Sur le même ensemble d’entraînement :

Seuil minimal de tissu	0	0,01	0,05	0,1	0,17	0,2	0,3
Tranches conservées	2560	953	783	631	353	61	0
Part du total	100 %	37 %	31 %	25 %	14 %	2 %	0 %

Ce mécanisme de filtrage a été expérimenté dans le notebook d’exploration, qui permet de visualiser cette distribution globalement et par sujet, et servira de base pour comparer l’effet du retrait des tranches vides (voire peu remplies) sur l’entraînement du modèle de segmentation.

5 Réduction du fond : crop à la bounding box du cerveau

Le problème des tranches vides identifié ci-dessus n’est qu’un symptôme d’un problème plus large : le dataset iSeg-2017 étant déjà *skull-strippé*, le fond hors du cerveau vaut exactement 0 sur les deux modalités T1 et T2, sur un champ de vue nettement plus grand que le cerveau lui-même (volume $144 \times 192 \times 256$ par sujet). Même les tranches non vides contiennent donc une large marge de fond qui n’apporte aucune information utile au modèle.

5.1 Vérification empirique

Sur les 10 sujets d’entraînement, le masque d’intensité obtenu par simple seuillage ($T1 > 0$ ou $T2 > 0$) est **identique voxel à voxel** au masque de fond du label (label $\neq 0$), sans une seule exception. Un simple seuillage d’intensité suffit donc, sur ce dataset, à isoler exactement le cerveau, sans risque de perte de tissu réel.

5.2 Implémentation

Pour chaque sujet, une bounding box 3D est calculée à partir de l’union des voxels non nuls de T1 et T2, avec une marge de sécurité de 4 voxels (tolérance pour un sujet légèrement atypique, en particulier les sujets du split test qui n’ont pas de label pour vérifier la bbox directement).

Comme chaque sujet a une bbox de taille différente, une taille de canvas commune à toute la cohorte est ensuite imposée — le maximum des tailles individuelles, arrondi au multiple de 8 supérieur pour rester compatible avec la profondeur (`depth=3`) du `CompactUNet`. Sur les 10 sujets d’entraînement, le canvas passe ainsi de 256×256 à 160×160 , et le nombre de tranches par sujet est quasiment divisé par deux (les tranches entièrement vides en bord de volume étant éliminées par le crop).

5.3 Impact sur l’entraînement

Une comparaison contrôlée (même seed, mêmes hyperparamètres — `base_channels=16`, `depth=3`, `lr=1e-3`, `batch_size=16`, 10 epochs, même split validation) entre le pipeline actuel et l’entrée cropée donne :

	sans crop	avec crop
Canvas	256×256	160×160
Tranches (train / val)	2048 / 512	874 / 216
Dice moyen (fg), meilleure epoch	0,822 (epoch 9)	0,841 (epoch 7)
Temps moyen par epoch	~ 110 s	~ 14 s

Le crop apporte un gain de vitesse d’environ $7,6\times$ par epoch, cohérent avec la réduction du nombre de pixels traités (canvas divisé par $\sim 2,6$, nombre de tranches par sujet divisé par ~ 2), ainsi qu’un léger gain de Dice (+0,019 sur la meilleure epoch, plus net sur la substance blanche) — un seul run par condition, donc pas une preuve de significativité statistique, mais cohérent avec l’hypothèse que retirer le fond trivial concentre le gradient sur les vraies frontières de tissu.

Mise en garde sur les valeurs de `train_loss/val_loss`. Ces deux quantités ne sont pas comparables entre les deux runs. `nn.CrossEntropyLoss()` moyenne la perte par pixel, pas par classe : sans crop, 95,2 % des pixels sont du fond, contre 71,4 % avec crop (mesuré via `BrainSliceDataset.foreground_ratio`). Le fond étant la classe la plus facile (Dice $\approx 1,0000$ dès l’epoch 5 dans les deux runs), sa contribution quasi nulle à la perte tire d’autant plus la moyenne vers le bas que sa part de pixels est grande — ce qui explique la perte plus faible observée sans crop (0,022 contre 0,116 à l’epoch 9), sans que le modèle soit pour autant meilleur. C’est un piège méthodologique classique des métriques moyennées par pixel sous fort déséquilibre de classe : le Dice foreground, moyenné par classe et non par pixel, reste la métrique valide pour comparer les deux runs.

5.4 Overhead du crop à l’inférence

Le gain observé à l’entraînement s’explique en bonne partie par l’amortissement du coût de calcul de la bbox — effectué une seule fois, à la construction du dataset — sur de nombreux batches et epochs. À l’inférence, un seul passage *forward* par sujet, la question se pose différemment : l’overhead de calcul de la bbox est-il compensé par le gain d’un forward pass plus petit ?

Une mesure directe sur un sujet (CPU, `CompactUNet` en mode évaluation, `torch.no_grad()`) isole le coût partagé (chargement des volumes, identique dans les deux cas), l’overhead spécifique au crop (seuillage et calcul de bbox) et le coût du forward pass :

Étape	Temps
Chargement T1+T2 (partagé)	0,4 ms
Overhead crop — seuillage + bbox	27,9 ms
Overhead crop — découpe	<0,1 ms
Forward pass sans crop (256×256, 256 tranches)	4623 ms
Forward pass avec crop (160×160, 115 tranches)	569 ms

L’overhead total du crop (environ 28 ms) ne représente que 0,7 % du gain apporté sur le forward pass (environ 4055 ms) : il est largement compensé. Le speedup net à l’inférence sur un sujet (overhead compris) est d’environ 7,75×, quasiment identique au ×7,6 observé par epoch à l’entraînement — signe que le déséquilibre de coût entre un seuillage numpy vectorisé sur des voxels et un réseau de convolutions sur des pixels est structurel, indépendant du nombre de répétitions.

Pour une inférence réelle sur le split test (sans label), il restera à replacer la prédiction dans le repère du volume d’origine (padding inverse de la bbox vers la forme d’origine) afin de l’exporter ou de l’évaluer — une opération d’un coût du même ordre de grandeur négligeable que la découpe elle-même.